

API SECURITY ASSESSMENT

crAPI — Controlled API Security Testing Lab

Prepared as a practical cybersecurity assessment project

Tools: Burp Suite • Docker • Firefox • crAPI

Environment: Local, isolated laboratory

Assessment focus: API authentication and object-level authorization

Prepared by: Pritam Bose

1. Executive Summary

This assessment was conducted against crAPI, a deliberately vulnerable API-focused application deployed in an isolated local laboratory. The objective was to move beyond basic web-page testing and examine how an API handles authentication, object identifiers, authorization decisions, and responses containing application data. The assessment used two separate test accounts and Burp Suite to observe the HTTP requests generated by the application. Video objects were created through the application and the corresponding API traffic was inspected. Object identifiers returned by the API were then examined and tested to determine whether access to an object was properly tied to the authenticated user's ownership. The most important result was an object-level authorization weakness consistent with Broken Object Level Authorization (BOLA). The testing demonstrated that changing an API object identifier could result in a successful HTTP response for another object. The finding is significant because API authorization must be enforced at the server side for every object requested; possession of a valid authentication token alone must not automatically grant access to every object identifier. The assessment also recorded normal authorization behavior on a protected vehicle endpoint, where an unauthorized operation returned HTTP 403. This response was treated as evidence of an existing access-control check rather than being incorrectly reported as a vulnerability. The distinction between a successful security control and a security weakness is an important part of the assessment methodology.

2. Scope and Objectives

The scope was limited to the locally deployed crAPI instance and the API functionality reachable from the test browser. No external systems were targeted. Testing was performed inside a controlled laboratory environment so that requests, accounts, and test data could be safely manipulated. The primary objectives were to understand the application's API structure, capture authenticated API requests, identify object identifiers and their associated responses, test whether object access was correctly authorized, and document the evidence in a reproducible manner.

3. Laboratory Environment

The application was deployed using Docker Compose. The deployment included the crAPI web interface and supporting services required by the application. The environment was verified before testing by checking that the relevant containers were running and healthy. Firefox was used as the application client and Burp Suite was used as the interception and request-analysis tool. Burp Suite allowed the HTTP method, endpoint, authorization header, parameters, response status, and returned JSON data to be examined directly.

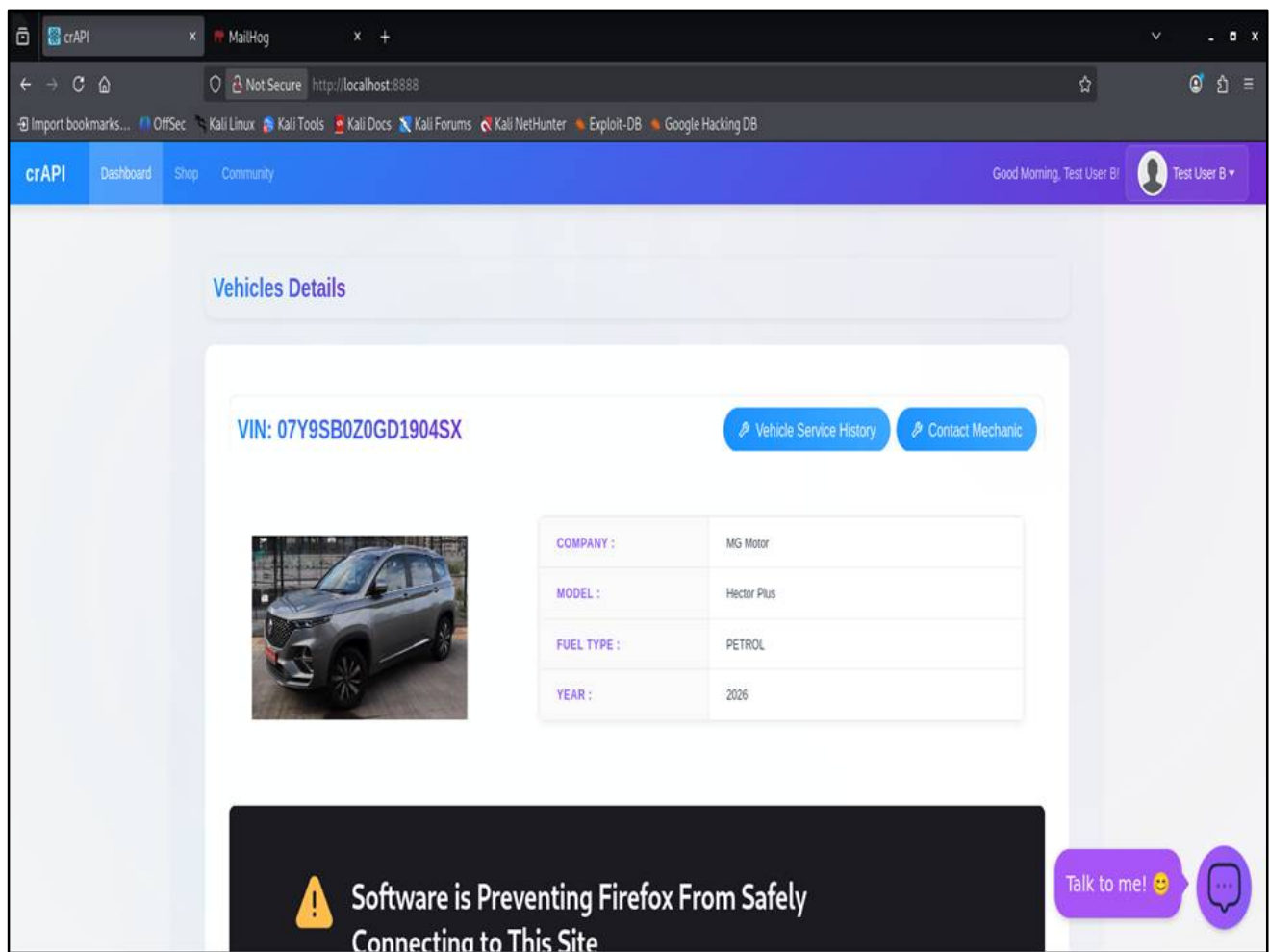


Figure 1. crAPI application dashboard. The authenticated crAPI dashboard used as the controlled API testing environment.

```

(kali@kali) ~/crAPI-main/deploy/docker
$ sudo docker compose ps

```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
api.mypremiumdealership.com	crapi/gateway-service:latest	"/app/server"	api.mypremiumdealership.com	3 minutes ago	Up 3 minutes (healthy)	443/tcp
chromadb	chromadb/chroma:latest	"dumb-init -- chroma..."	chromadb	3 minutes ago	Up 3 minutes (healthy)	8000/tcp
crapi-chatbot	crapi/crapi-chatbot:latest	"/bin/sh -c /app/ent..."	crapi-chatbot	3 minutes ago	Up 2 minutes	5002/tcp, 5500/tcp
crapi-community	crapi/crapi-community:latest	"/bin/sh -c /app/main..."	crapi-community	3 minutes ago	Up 2 minutes (healthy)	6060/tcp
crapi-identity	crapi/crapi-identity:latest	"/_cacert_entrypoint..."	crapi-identity	3 minutes ago	Up 2 minutes (healthy)	8080/tcp, 8989/tcp, 10001/tcp
crapi-web	crapi/crapi-web:latest	"/etc/nginx/nginx-wr..."	crapi-web	3 minutes ago	Up 59 seconds (healthy)	127.0.0.1:8888→80/tcp, 127.0.0.1:30080→80/tcp
crapi-workshop	crapi/crapi-workshop:latest	"/app/runner.sh"	crapi-workshop	3 minutes ago	Up About a minute (healthy)	
mailhog	crapi/mailhog:latest	"MailHog"	mailhog	3 minutes ago	Up 3 minutes (healthy)	1025/tcp, 127.0.0.1:8025→8025/tcp
mongodb	mongo:4.4	"docker-entrypoint.s..."	mongodb	3 minutes ago	Up 3 minutes (healthy)	27017/tcp
postgresdb	postgres:14	"docker-entrypoint.s..."	postgresdb	3 minutes ago	Up 3 minutes (healthy)	5432/tcp

```

(kali@kali) ~/crAPI-main/deploy/docker
$

```

Figure 2. Docker laboratory status. Docker Compose output showing the crAPI services running in the local assessment environment.

4. Assessment Methodology

The assessment followed a request-driven workflow rather than relying only on the graphical interface. First, the application was accessed with a test account and normal application actions were performed. Burp Suite was used to capture the resulting API requests. The captured requests were then reviewed to identify endpoints, HTTP methods, authentication tokens, object identifiers, response codes, and the information returned by the server. Two test accounts were used during the object-level authorization assessment. Test data was created through the respective accounts so that ownership could be distinguished. The API requests generated by these actions exposed the identifiers assigned to the created objects. Those identifiers were then used as controlled test inputs while keeping the assessment within the local crAPI environment. The core authorization question was simple: if User A is authenticated, does the server verify that the object requested by User A actually belongs to User A? If the answer is no, an attacker who learns or guesses another object's identifier may be able to access that object without legitimate authorization.

5.1 Broken Object Level Authorization (BOLA)

Send Cancel

Request
Response

Pretty Raw Hex
 In

```

1 GET /identity/api/v2/user/videos/52 HTTP/1.1
2 Host: localhost:8888
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
4 Accept: */*
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Referer: http://localhost:8888/my-profile
8 Content-Type: application/json
9 Authorization: Bearer
  eyJhbGciOiJIUzI1NiIsInR5cCI6IkpzZW50L3VudCcpEA39m-SNlUeOkq01EvFr4jtr-LJSCh0Vx_pb5nEoW8P32d
  tLpLaMAA6xVdClu6FiIgv3LPY6-y27LZPmatVgdIi_z-DaOhJBm08R6JtiZJcuP4LVjpwOENHFKFPX580vie6Q
  XDUK45yPq9KGzZWmNd4sQ0r4h5MPZn-y2L0t9M4Rz0J2yieQ5QZ457q6J5L292fnsqpgL08tRlUjOngPCia
  JY3n_q6PH7rJY8-iaEGL2mxkyqiATJo9H672xL4fUzYj8mA
10 Connection: keep-alive
11 Cookie: language=en; welcomebanner_status=dismiss; continueCode=
  wWqZLP8MrgezNnoRk4EQ37ABX17S88hmRcLoGSJxyvjD9OKYabpZX68n1wL
12 Priority: u=4

```

```

1 HTTP/1.1 200
2 Server: openresty/1.27.1.2
3 Date: Thu, 27 Aug 2026 17:45:18 GMT
4 Content-Type: application/json
5 Connection: keep-alive
6 Vary: Origin
7 Vary: Access-Control-Request-Method
8 Vary: Access-Control-Request-Headers
9 X-Content-Type-Options: nosniff
10 X-XSS-Protection: 0
11 Cache-Control: no-cache, no-store, max-age=0, must-revalidate
12 Pragma: no-cache
13 Expires: 0
14 Strict-Transport-Security: max-age=31536000 ; includeSubDomains
15 X-Frame-Options: DENY
16 Content-Length: 216734
17
18 {
  "id":52,
  "video_name":"Screenshot 2026-05-07 234617.png",
  "conversion_params":{"v_codec h264",
  "profileVideo":
  "data:image/jpeg;base64,iVBORwOKGgoAAAAANSUHEuJgAAAXBAAAGLCAYAAAAvRSJoAAAAAXNSI
  Ars4c6QAAAAARnQU1BAACxjwv8YQUAAAAJCehZcwAADsMAAA7ACdvqQQAAP+lSURBVHhe7H13oCR
  evenZ3X7y50+mk33TKOSIBAgSIIiWGXhM8mc+LaxBt sY/GfjwPCBC044YTDGkWyNggJESWhdAq
  unudDm+vGm+/ujqnp65s3uzr737iT8+Xe3b3d6quqrqur4/QOAIceICI450DEI65nbIam6QaPi
  wYRk2/c+gji96PVDkfciRfC6KEcZevFfzGsOFFlCBX7KdAVJBzef3L1hxyrsDmgYTM8lAhhBJ2kDp
  Vm+bZS/9LgbJyUJTDKwsroqpcYKqORhMb5eh6S/S8rOXSWjGkekqCshLM8izd405p0rTdtLnrf
  IcZbOhT7x7wLlF32ilvgjaf29skjHCqHh9SqbBhB34NmStSy9nH5ve4pe/0i9XnHKLUDSGHAYL1k
  VLF634oi98tLSTIVZZ+MehFr9e9Majdoku6I3idRXOS9PvflX0s2PSHGhLfrLVP0VT2+GL+Y00S
  3vt9GI4mk69KfZMPG6DFXihAiLcdCORwLZaJboS8lelW0Mv7DsMxyV3SII0YJ0Q0ueh6AkK9xSe
  WUNWAF6pD2MhzbCsuzl+2GKNIooqdfjSOBqrkGfLeK343e+2ftquv9MPTHqsfLaUFHYv8qhhl
  jSfpa6NOLBfa9mLgUVZVuFjDYIymt30WhZKxbvXK6KiSprjAJLVRwArxxRS++bvsmq4nPrZCw
  oiv5VzXHOFGYfhqF5hmOSKrmC/SPXCzT3A15VxmKdVZsQL4sUQVbqRBWZ6d8VRT60VsvRFPmRr
  F8wzFPnnuiJDJR4eJihvYxgcRNB/wX8v9KLbLsjXlcePdj4i07GehuhJV9dipDIUaZSLJQcmEdYw
  t+Tetix0aiHPRR3KMH464W44XAI5fH77nItf2nGK6cPrhoRbMnaFlR7Fnuah331vH0WCKH0

```

Request	Response
<pre> 1 GET /identity/api/v2/user/videos/53 HTTP/1.1 2 Host: localhost:8888 3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0 4 Accept: */* 5 Accept-Language: en-US,en;q=0.5 6 Accept-Encoding: gzip, deflate, br 7 Referer: http://localhost:8888/my-profile 8 Content-Type: application/json 9 Authorization: Bearer eyJhbGciOiJSUzI1NiJ9.eyJzdWIiOiJlci2VyYkBlcGtGtGxLmNvbSImlhdCI6MTc4NDZlMzAwMSwiZXh wIjoxNzg4NDU3ODAxLjY2b2xlIjoidXNlciJ9.d3ruKEoV6HyKn9xpGp_nWSoWSW-b3VZzHOB9fmJZYv6S 2fh1t40_qQ3N7Wl-3Qbl07-kCFWN2VGhZ_UDm4JVi0-XpDx7anh8uzmJThikbY5le8SMRwiDsgG1xI6Vczk 2TotTDz1-TD7cV1DVuYlXQ06JZtsczR0iz9RemDCfaKQuxF9AF3YicSe-t9XyJz0kjTGgrWaJHlcJ8-CAfyj qp7cgRBinuBinVvDGIbH3K1s_rkK027sWw-W8Zp_5NCcxEdxPcQyubk5WMczE7tNfa8YtQh65EnkwYIBRhaP 8v7e9NaC6hZcK61Ld_ajolQqBK_Hnxtl_FX2NBY9KPg 10 Connection: keep-alive 11 Cookie: language=en; welcomebanner_status=dismiss; continueCode= wWq2LVP6MrgezNmoRk4EQ37ABXI7S88hmRcLoG5JxvyjD9OKYabpZX6Bnlw1 12 Priority: u=4 </pre>	<pre> 1 HTTP/1.1 200 2 Server: openresty/1.27.1.2 3 Date: Thu, 27 Aug 2026 17:54:59 GMT 4 Content-Type: application/json 5 Connection: keep-alive 6 Vary: Origin 7 Vary: Access-Control-Request-Method 8 Vary: Access-Control-Request-Headers 9 X-Content-Type-Options: nosniff 10 X-XSS-Protection: 0 11 Cache-Control: no-cache, no-store, max-age=0, must 12 Pragma: no-cache 13 Expires: 0 14 Strict-Transport-Security: max-age=31536000 ; incl 15 X-Frame-Options: DENY 16 Content-Length: 85230 17 18 { "id":53, "video_name":"Screenshot 2026-05-07 233650.pn "conversion_params":"-v codec h264", "profileVideo": "data:image/jpeg;base64,iVBORwOKGgoAAAANSUHEU Ars4c6QAAAAARnQU1BAACxjvw8YQUAAAACJcEhZcwAADsMA Z/e59wY+WqznFyTmQYBxySJBuzKhHARBH1Uz/lEzALJhQ nP2Xt8fa5/qZhhcg/S63nqqbpVt+69Z5/9rviutU2WZc chyUD7sci+jgWLnzcXmo5Dgov608VBvb3PMXj4Aci/c8L Py/erHAFloiqOhbU9F95b/JgPseD+d9jLw8zKL+3F+fy n2gDDmWN6/Y/nex1YeZlAelwcu4ZNeewVw/ejHAF1cfk B6X7yDHY8pjIcdB+W3L+IY8HLMeGzkOym8rxzfkTk2ch /8fle1EeZlAel/IPXI6v3ferPMYgPK7LH7iY48D8PpWHG 9+IiYe/ndcfl+k00gfFjlm0F18Ige8U6/i8d7t5Jp1ecf YggiAggvC0H0CpYoiCDDYKMEY1ZXGKJClheo40r9f5Dg tJjAGstUVIhSsoYE4ExGGMvM5jbHTPdzwu/OvLOCI/nY </pre>

Figure 4. Second object access test. Burp Suite showing GET /identity/api/v2/user/videos/53 with HTTP 200 and the returned JSON object.

Impact: In a real application, an equivalent authorization weakness could allow one authenticated user to access another user's private resources by changing an object identifier. The impact depends on the sensitivity of the object represented by the endpoint. In this lab, the test was limited to controlled video objects and did not target real users or production data.

Root cause: The server-side authorization decision must be bound to both the authenticated principal and the requested object. Checking only whether the requester possesses a valid token is insufficient when the request contains a user-controlled object identifier.

Recommended remediation: The API should retrieve the requested object together with its ownership information and verify that the authenticated user's identifier matches the object's owner before returning the resource. Authorization should be enforced on every object-level read and write operation, not only in the front-end interface.

5.2 Authorization Control Observation

A separate request to the vehicle-addition API returned HTTP 403 Forbidden. This behavior indicates that the endpoint performed an authorization check and rejected the tested operation. This result was deliberately recorded as a security-control observation rather than a vulnerability finding. A 403 response by itself is not evidence of a security flaw.

#	Host	Method	URL	Params	Edited	Status code	Length	MIME type	Extension	Title	Notes	TLS	IP	Cool
76	http://localhost:8888	GET	/identity/api/v2/vehicle/vehicles			200	852	JSON					127.0.0.1	
78	http://localhost:8888	POST	/identity/api/auth/verify		✓	200	571	JSON					127.0.0.1	
79	http://localhost:8888	POST	/identity/api/auth/signup		✓	200	584	JSON					127.0.0.1	
80	http://localhost:8888	POST	/identity/api/auth/login		✓	200	1056	JSON					127.0.0.1	
81	http://localhost:8888	GET	/identity/api/v2/user/dashboard			200	673	JSON					127.0.0.1	
82	http://localhost:8888	POST	/identity/api/auth/verify		✓	200	571	JSON					127.0.0.1	
83	http://localhost:8888	GET	/identity/api/v2/user/dashboard			200	673	JSON					127.0.0.1	
84	http://localhost:8888	GET	/identity/api/v2/vehicle/vehicles			200	483	JSON					127.0.0.1	
93	http://localhost:8025	GET	/api/v2/websocket			101	129						127.0.0.1	
94	http://localhost:8025	GET	/api/v1/messages/VPSNRqVLUakr6U...			200	5326	JSON					127.0.0.1	
95	http://localhost:8888	POST	/identity/api/v2/vehicle/add_vehicle		✓	403	578	JSON					127.0.0.1	
96	http://localhost:8025	GET	/			200	18555	HTML		MailHog			127.0.0.1	
118	http://localhost:8025	GET	/api/v2/jim			404	82						127.0.0.1	
119	http://localhost:8025	GET	/api/v2/messages?limit=50		✓	200	6556	JSON					127.0.0.1	

Request

Response

Pretty Raw Hex

🔍

📄

In

≡

1 POST /identity/api/v2/vehicle/add_vehicle HTTP/1.1

2 Host: localhost:8888

3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0

4 Accept: */*

5 Accept-Language: en-US,en;q=0.5

6 Accept-Encoding: gzip, deflate, br

7 Referer: http://localhost:8888/verify-vehicle

8 Content-Type: application/json

9 Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpzZW50L3NpdjkiLCJ1eWUiOiJlc2VybklleGlGPTc4LnNbSIsImhhdCI6IjE4Mjc4NDZlMzAwMSwiZXNwIjozNg40NDU3ODAxLCJyb2xiOioidXRyYkxkLmN1ciJ9.d3ruKEoV6HyKn9xpGp_nWSowSW-b3VZzH0B9fmJZYv6S2fhIt40_q3N7mL-3QblO7-kCFNN2VGhZ_Udm4JVio-XpDx7anh8uzmJThikBYSlE8SMRWIdsgGlxIE6vcz2TotTdZl-TDTcVLDvuYLX006JztsczRoiz9RemDCfaKQuxF9AF3YicSe-t9XyJzOkjTGgrWaJHlcJB-Cafyjqp7CgPBinuBinVdGIbh3Kls_rkKO275Ww-W8Zp_5NCcxEdxPcQyubk5WMczE7tNfa8ytOh6SEEnkvYIBPhAP8v7eNaC6hZcK6Lld_ajolQdQBK_Hnxtl_FX2NB9YKpg

10 Content-Length: 44

11 Origin: http://localhost:8888

12 Connection: keep-alive

13 Cookie: language=en; welcomebanner_status=dismiss; continueCode=wIQ2LP8MrgezNm0rkEQ37ABXI7S88hmRcLoGSjxyvjD9OKYabpXZ6Bnlw1

14 Priority: u=0

15 {

16 }

Pretty Raw Hex Render

🔍

📄

In

≡

1 HTTP/1.1 403

2 Server: openresty/1.27.1.2

3 Date: Thu, 27 Aug 2026 17:52:06 GMT

4 Content-Type: application/json

5 Connection: keep-alive

6 Vary: Origin

7 Vary: Access-Control-Request-Method

8 Vary: Access-Control-Request-Headers

9 Access-Control-Allow-Origin: *

10 X-Content-Type-Options: nosniff

11 X-XSS-Protection: 0

12 Cache-Control: no-cache, no-store, max-age=0, must-revalidate

13 Pragma: no-cache

14 Expires: 0

15 Strict-Transport-Security: max-age=31536000 ; includeSubDomains

16 X-Frame-Options: DENY

17 Content-Length: 64

18

19 {

20 "message": "Sorry, This Vehicle is already added.",

21 "status": 403

22 }

Figure 5. Authorization rejection. Burp Suite showing the protected vehicle endpoint returning HTTP 403 Forbidden.

6. API Security Observations

The assessment demonstrated why API testing requires examining the underlying HTTP traffic instead of relying only on what the web interface displays. The crAPI interface did not provide a convenient visual representation of the uploaded video objects in the profile view, while Burp Suite exposed the actual API requests and JSON responses. The captured requests also showed the use of Bearer authentication tokens. These tokens are security-sensitive and should not be published in a public portfolio repository. Screenshots intended for GitHub or LinkedIn should therefore be redacted before publication.

7. Commands and Operational Steps

The laboratory was operated locally using Docker Compose. The following command was used to verify the state of the deployed services:

```
sudo docker compose ps
```

After the services were running, the crAPI web interface was opened locally. Two controlled test accounts were used. Video data was uploaded through the application, and Burp Suite was used to capture the resulting API traffic. The relevant GET requests were then sent through Burp Repeater so that object identifiers could be examined and tested.

The important operational principle was to modify only the object identifier under test while preserving the authenticated request context. This makes the test meaningful because it evaluates whether the server independently checks ownership instead of merely testing whether the endpoint exists.

8. Evidence Interpretation

Figure 1 establishes the application context, while Figure 2 establishes the local deployment state. Figures 3 and 4 provide the core API evidence for object-level testing. Figure 5 demonstrates that the application can also reject an unauthorized operation with HTTP 403. The strongest evidence in this assessment is the combination of authenticated API requests, controlled object creation using separate accounts, and successful object responses. The report therefore does not treat every unusual response as a vulnerability. The 403 response, for example, represents a functioning authorization control for that tested operation.

9. Remediation and Retesting Strategy

A secure implementation should associate every protected object with an owner or authorization policy and enforce that policy on the server before returning or modifying the object. The authorization decision should be based on the authenticated identity extracted from the session or token rather than on a user identifier supplied by the client.

After remediation, the same request sequence should be repeated. A user should be able to retrieve their own authorized object, while attempting to access another user's object should return an appropriate denial such as HTTP 403 or an equivalent authorization failure. The important point is that changing an object identifier alone must no longer grant access.

A complete retest should also verify both positive and negative cases: legitimate access must continue to work, while cross-user access must consistently fail across relevant read and write endpoints.

10. Limitations

This was a controlled laboratory assessment against crAPI and not a production penetration test. The evidence demonstrates behavior within the configured local environment. The assessment did not attempt to access real users, external systems, or production information. The absence of a visible video in the crAPI profile interface was not treated as evidence that the API object did not exist. API-level Burp Suite evidence was used because it directly represented the server request and response.

11. Conclusion

This project demonstrated a practical API security assessment workflow using a deliberately vulnerable application rather than a collection of isolated tool commands. The work began with deployment of crAPI in a controlled Docker environment and continued through authenticated application use, API interception, request analysis, object-identifier testing, and interpretation of server responses.

The central security lesson from the assessment is that authentication and authorization are different controls. A valid Bearer token establishes an authenticated identity, but it does not automatically establish permission to access every object exposed by an API. The object-level authorization test showed why the server must make an explicit ownership or permission decision whenever a client requests an object by identifier.

The project also demonstrated disciplined evidence handling. Successful HTTP 200 responses were used where they represented meaningful object access, while the HTTP 403 response was not misclassified as a vulnerability. This distinction matters in professional security work because a credible assessment must separate confirmed weaknesses from normal security behavior.

Overall, the assessment provided practical experience with API request structure, Bearer-token authentication, object identifiers, HTTP status codes, Burp Suite Repeater, Docker-based lab deployment, and the BOLA class of authorization weakness. The same methodology can be transferred to other APIs: map the endpoint, understand the authentication context, identify the object reference, test the authorization boundary, document the response, explain the root cause, recommend a server-side fix, and retest the exact attack path.

12. References

OWASP API Security Top 10 — Broken Object Level Authorization (BOLA).

PortSwigger Web Security Academy — Access control and authorization testing concepts.

crAPI — Completely Ridiculous API, a deliberately vulnerable API security training application.

Burp Suite Documentation — HTTP interception, Repeater, and request/response analysis.